24 March 2026

Loop Invariants: the tool we use to prove that a loop (algorithm) works.

The Ingredients:

- The loop guard, G, is what keeps us in the loop.
  While stmt: just copy the boolean stmt
  for loop: rewrite as a while loop
- The post condition, Q: what we hope the loop will achieve / the goal.
  usually, from our problem statement
- The preconditions, P: what is true before entering the loop. State Everything connected by "and"s.
- The loop invariant, L: a statement that INVARIANT we could put in an assert() every time we attempt to re-enter the loop, and something that "builds up" to Q. → how the proof part works.
  (sometimes: a weaker version of Q, maybe)

The courses of the LI "meal"

① INITIALIZATION $G \wedge P \Rightarrow L$

analogy: induction base case
idea: first time we attempt to enter the loop, the loop invariant is true.
proof: use G + P exclusively to get to L

② MAINTENANCE $\underbrace{L_i}_{} \wedge G \Rightarrow L_{i+1}$

L_i ↳ L at the ith time I attempt to enter the loop

~~idea~~ analogy: assuming L_i = I.A in induction
proving L_{i+1} = inductive step
idea: everytime I enter the loop, I may intermediately break things, but by the time I'm done, L is true again.
proof: Assume L_i + G. walk through the lines of (pseudocode) in the loop. At the end, you should be able to conclude L is true (again).
* It is ok for LI to break, as long as it gets fixed.

③ END $\neg G \wedge L \Rightarrow Q$

idea: at the end of the loop's execution, the loop did what it was supposed to do.
proof: use ¬G + L and nothing else to get to Q

DNF (A) an array of length n


```
i = 1
last red = 0
firstblue = n+1
while i < firstblue
    if A[i].color = red
        lastred ++
        Swap A[i] and A[lastred]
        i++
    elseif A[i].color = blue
        firstblue --
        Swap A[i] and A[firstblue]
    else
        i++
    end if
end while
return A
```

DNF: the statements

G = "$i <$ firstblue" ; $\neg G$ = "$\neg(i <$ firstblue$)$" $\Leftrightarrow i \geq$ firstblu

Q = A is now sorted red, then white elts, then blue elts

A = [red*, white*, blue*]

P = A is an array of size n of elts each of which is red, white or blue (random order)

and

$i = 1$

and

lastred $= 0$

and

first blue $= n+1$

L = A[1 ... lastred] contains only red elements $=: L^R$

and

A[firstblue ... n] contains only blue elements $=: L^B$

and

A[lastred + 1 ... $i - 1$] contains only white elements $=: L^W$

and

i is an integer $\leq$ firstblue

INIT $P \Rightarrow L$

$A[1 \ldots lastred]$ is empty ($lastred = 0$)

$A[firstblue \ldots n]$ is empty ($firstblue = n+1$)

$A[lastred+1 \ldots i-1]$ is empty ($lastred+1 = 0+1 = 1$, $i-1 = 1-1 = 0$)

} All 3 sub-arrays are empty $\Rightarrow$ stmts L^R, L^B, & L^W are vacuously true.

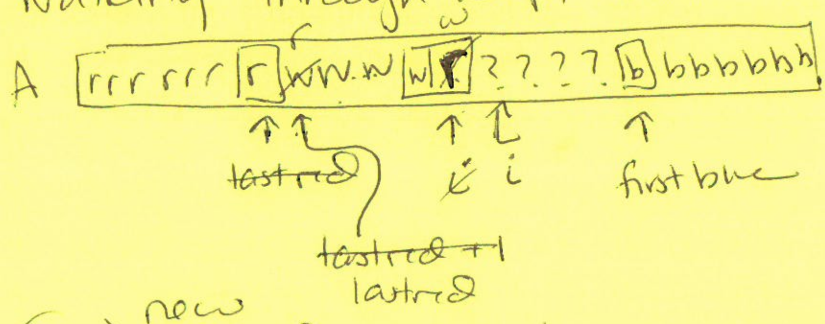
MAINT

Assume $i \geq 1$ and we are entering the ~~one~~ loop for the ith time and both G and L are true.

There are 3 cases:

Case 1: $A[i]$ is red.

Walking through loop, enter if on Line 5



(1a) new lastred is w:

line 7 swaps r & w elements

$A[1 \ldots lastred]$ is again all red

$A[lastred+1 \ldots i-1]$ is ~~unchanged other than~~ the same set as before (but 1st elt became the last)

$\Rightarrow$ still all white

L^B never touched $\Rightarrow$ still the same

(1b) new lastred = i, swap is effectively a noop. Same arg for L^R, L^B

(1b) same arg for L^R, L^B.

L^W was at top of loop an empty array & still is. So, still holds!

Lines 9-14 are skipped/not executed.

<u>Case 2:</u> A[i] is blue
<u>Case 3:</u> A[i] is white

} try these yourself!

END: $\neg G \wedge L \Rightarrow Q$

Assume $\underbrace{\neg(i < \text{firstblue})}_{\Downarrow}$ and $L^R \wedge L^B \wedge L^W$

$i \geq \text{firstblue}$

By L^R, A[1... lastred] are all red.

By L^B, A[firstblue ... n] are all blue.

By L^W, A[lastred +1 ... i-1] are all white

Because $i \geq$ firstblue (by $\neg G$) and $i \leq$ firstblue (by L)

$\Rightarrow i =$ firstblue

Thus A[lastred+1 ... firstblue-1] are all white.

And, the array is in an order consistent w/ Q.